PM101 · INTERACTION DESIGN · CANCELLATION EXPERIENCE

Teammates:
L Shailender Goud
Mehak Makol
Aashi Bhansali
Harshit Chebolu
Tanushree Mishra

Staymint

Own the clarity of cancellation.

Mobile wireframes for cancelling an existing hotel booking, designed around Agoda's cancellation APIs.

PROTOTYPE
Interactive synthetic demo:
7 screens, 9 reviewer scenarios

DELIVERABLE
7 wireframe screens in 12 selected views, with design, developer, business notes

DEMO DATA
Deterministic and synthetic:
Azure Palms Resort & Spa

ASSUMPTIONS

Android app. A booking created in Staymint is created with Agoda at once. Support is an internal booking-linked channel, not an Agoda call.

The Agoda pages referenced for this flow do not publish HTTP verbs, endpoint paths, reference expiry, or the exact check-in cutoff time; stated as assumptions where used.

October 2026

Product Note - the problem and the goal

What Staymint is fixing, for whom, and how big the problem is.

WHAT IT IS - PROJECT AND GOAL

> *Project: Staymint - the cancellation experience for our multi-hotel booking app, built on Agoda's inventory.*
> *Goal: cancel one existing booking knowing the refund before you commit, and the true status after.*

PROBLEM - WHAT AND FOR WHOM

FOR WHOM - THE CUSTOMER

A Staymint customer with one existing hotel booking who needs to cancel it - plans changed, zero-refund, amount moved at commit, unclear outcome, or support.

WHAT GOES WRONG TODAY

The refund consequence is unclear before they commit; an uncertain outcome can look like success; support restarts without the booking context.

HOW BIG IS THE PROBLEM?

India: the National Consumer Helpline recovered over Rs 10 crore in Travel & Tourism refunds, 25 Apr 2025-31 Jul 2026 [1]. That is money recovered through grievances, not total customer loss, and covers all travel, not only hotels.

For hotels, a room-night is perishable: once the night passes, it cannot be sold [2], so late cancellations leave less time to resell it.

Staymint targets the customer's uncertainty (refund amount, true status), not penalties or hotel occupancy.

Evidence behind the problem size

India refund grievances, what they show, and what stays unmeasured. Sources cited.

THE CUSTOMER'S PAIN

After cancelling, the customer needs two clear answers: "Is my booking cancelled?" and "How much money will I get back?" If the refund changes or confirmation fails, they may be unsure whether they are still booked or charged. That is the uncertainty our flow addresses.

Our reading, not tested with users: Agoda's cancel flow can cause this. The refund amount can move between preview and confirm (error 914), and a timeout leaves the status unknown [3].

THE COST, KEPT SEPARATE

Customer: a penalty the booking policy allows is a cost, not a failure. A delayed refund is money tied up, not lost. A wrong or wrongly denied refund is a loss.

India evidence: the National Consumer Helpline recovered over Rs 10 crore in Travel & Tourism refunds, 25 Apr 2025-31 Jul 2026 [1]. This is money recovered through grievances, not total customer loss, and it covers all travel, not only hotels.

Hotel: a room-night is perishable. Once the night passes, an unsold room's revenue is gone [2], so late cancellations leave less time to resell the room. Resale or penalties can offset the loss. We make rules, amounts and outcomes clearer; we claim no effect on occupancy.

SOURCES

[1] Press Information Bureau (PIB), "National Consumer Helpline facilitates refunds of over Rs 105 crore to consumers in 15 months", 9 Sep 2026. https://www.pib.gov.in/PressReleasePage.aspx?PRID=2308386&lang=1&reg=3

[2] Peaqplus, "What is Perishable Inventory in Hotels?" (vendor definition). https://peaqplus.com/learn-perishable-inventory.html

[3] Agoda Demand API documentation, "Cancel bookings". https://developer.agoda.com/demand/docs/cancel-bookings

Product Note - how we answer it

Objectives, approach, proof, and what is still unknown.

OBJECTIVES - WHAT GOOD LOOKS LIKE

1

Latest terms first

See the latest refund estimate before you confirm; if it moves, we ask again.

2

No fake certainty

An uncertain outcome is never shown as a success.

3

Context into support

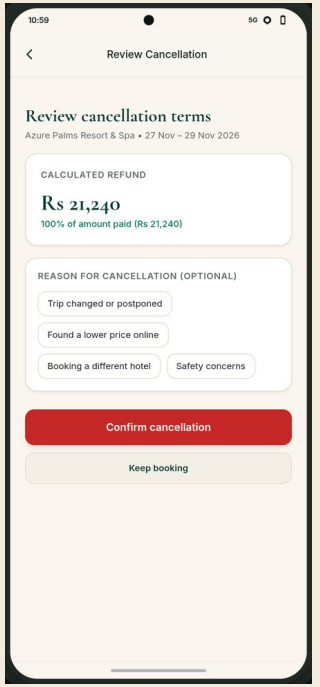
Support picks up where the customer left off - no starting over.

HOW WE ARE SOLVING IT

Two-step cancel on Agoda's APIs: Preview shows the refund and policy; Confirm sends the same reference and amount with a reason. Amount moved (error 914): re-preview, show old and new, take fresh consent. Timeout: no blind retry - recheck status, then resolve, retry, or escalate. Support is internal and carries the booking context. Scope: one existing booking only.

Trade-off accepted: extra checks over fewer taps - never friction to discourage cancelling.

CONCEPT MOCK



W3 Review cancellation - refund before commit.

Full set: Slides 8-12 · live demo: Slide 13.

SUCCESS METRICS - TO MEASURE, NOT CLAIMED RESULTS

- Refund disputes per 100 cancellations.
- 'What is my status' contacts per cancellation.
- Repeat-booking rate among customers who cancelled.
- Resolution time for cancellation support tickets.

OPEN QUESTIONS

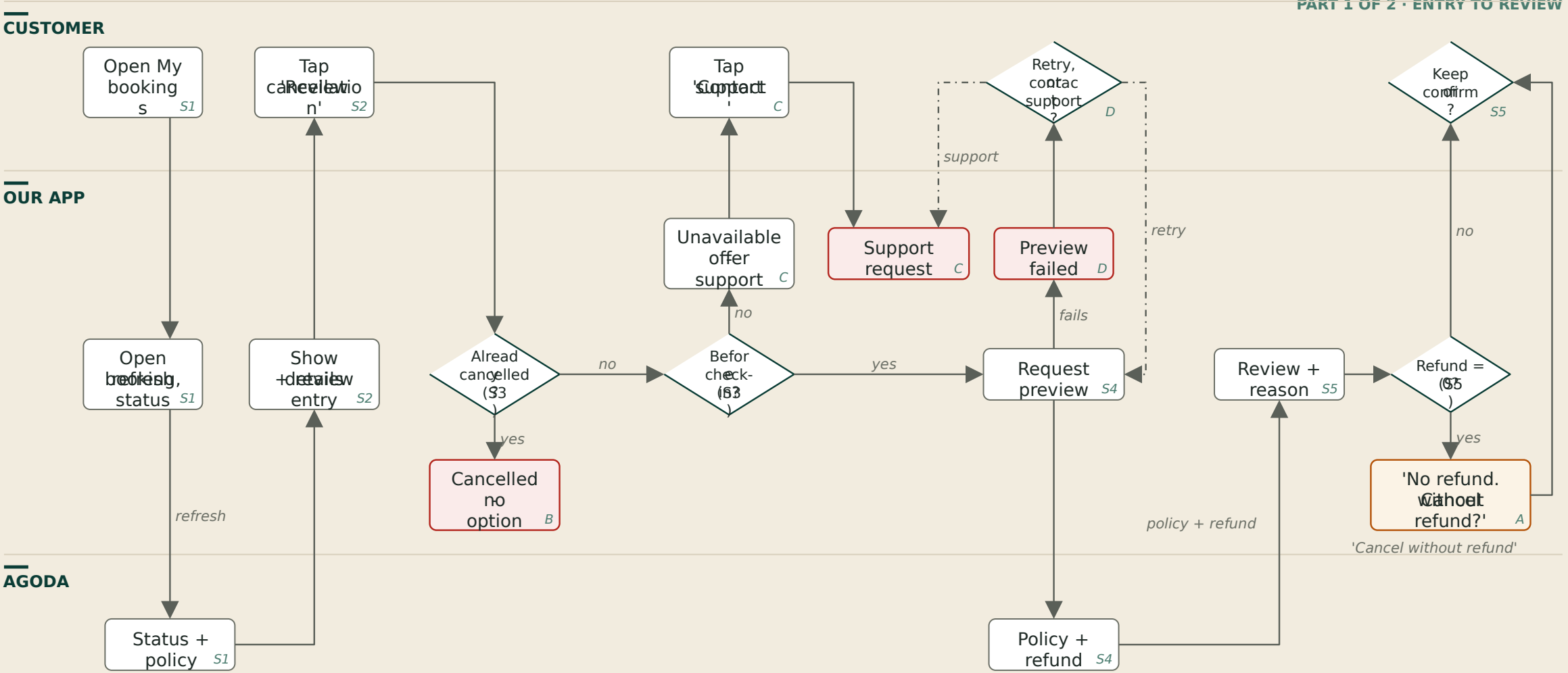
- Refund payout timing and method: not specified in the referenced docs; the receipt confirms cancellation only.
- Exact check-in cutoff, reference expiry: not specified. Fee-waiver wait: docs allow polling up to 72 h - when should we escalate to support?

The cancellation journey, part 1: entry to review

From opening a booking to the cancel review, including the unhappy paths. The keep-or-confirm decision continues in Part 2 (Slide 5).

decision continues on Slide 5

PART 1 OF 2 · ENTRY TO REVIEW

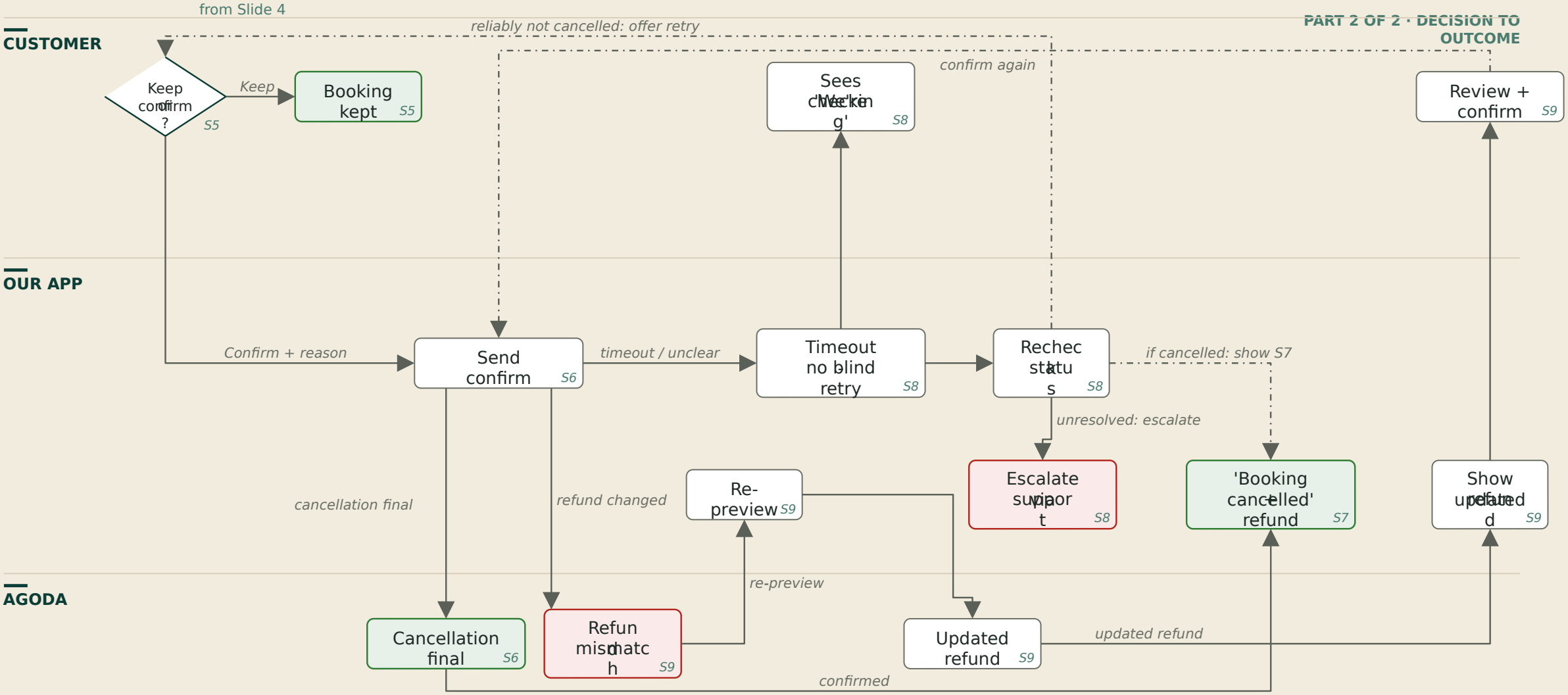


Diamond = decision · red = stop state · green = final state · dashed = return or recheck path

S1-S9 = journey step ids · A-D = exception branches · W1-W7 = wireframe screens (Slides 8-12)

The cancellation journey, part 2: decision to outcome

Continues from Slide 4: the confirm, the timeout reconciliation, and the changed-refund path that asks again.

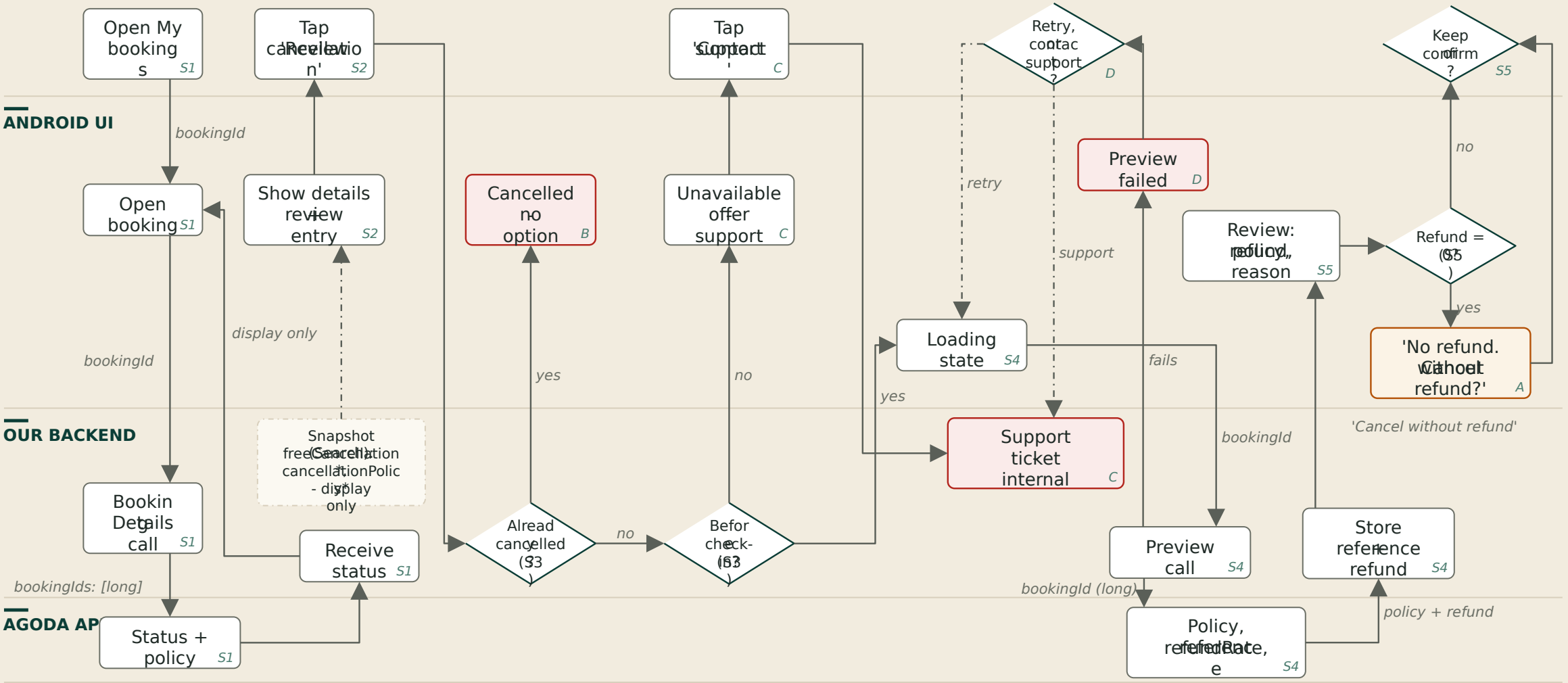


Diamond = decision · red = stop state · green = final state · dashed = return or recheck path

S1-S9 = journey step ids · A-D = exception branches · W1-W7 = wireframe screens (Slides 8-12)

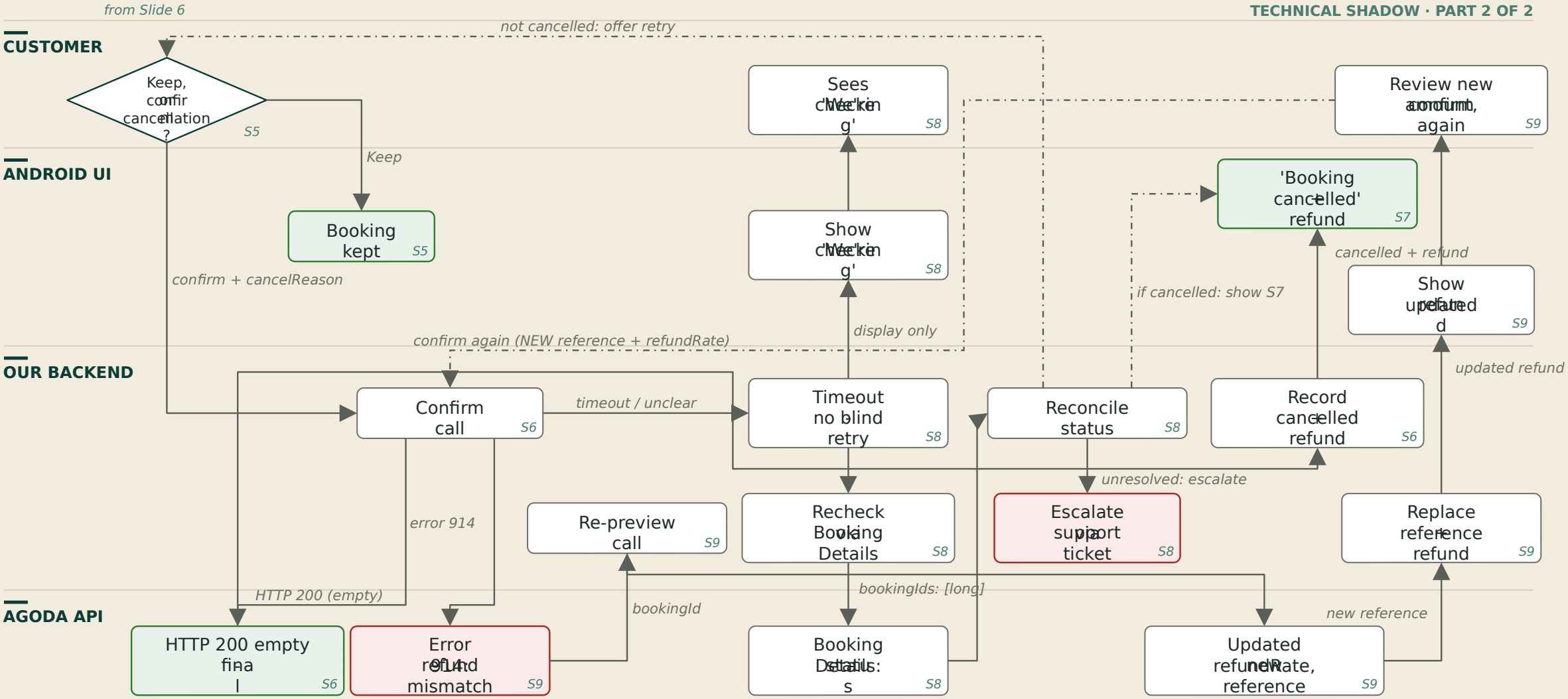
How the system keeps the promise, part 1: entry

The same journey as Slide 4, drawn as calls between the customer, the Android UI, our backend and the Agoda API. Part 2 (confirm, reconcile, outcomes) is on Slide 7.



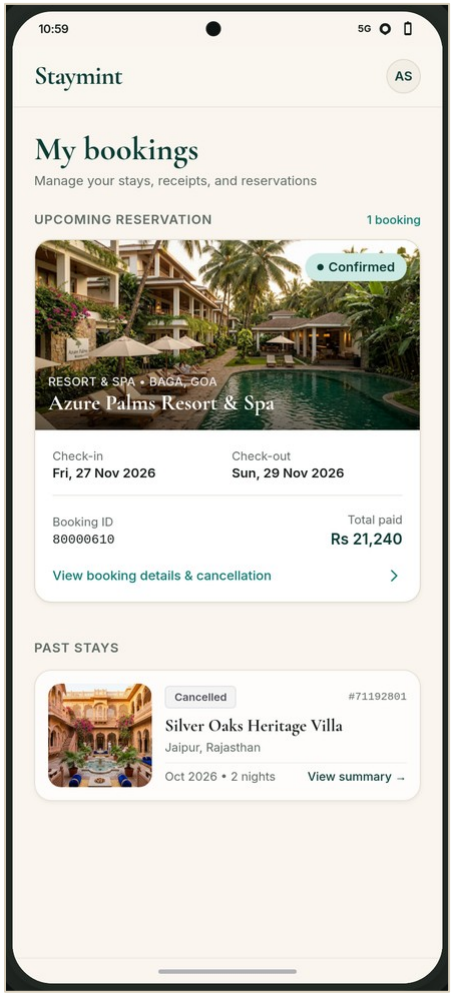
How the system keeps the promise, part 2: outcomes

Continues from Slide 6: the confirm call, timeout reconciliation, and the changed-refund path (error 914).



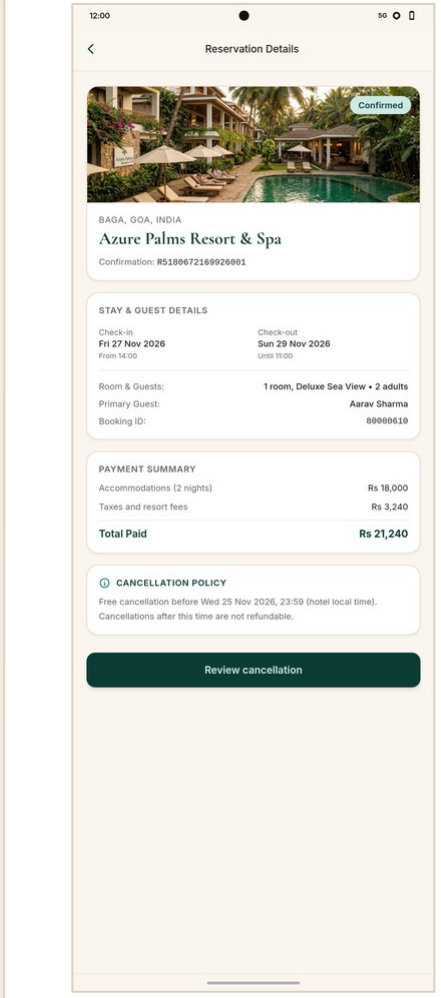
Wireframes 1-2: My bookings → Reservation details

Booking status is visible before anyone asks for it.



W1

My bookings - status at a glance



W2

Details - cancel entry

DESIGNER

W1 Visibility of system status - status chips show state before anyone asks, so there is no digging.

W2 Shallow IA - review entry is one tap from details, so the path to cancel involves no hunting.

DEVELOPER

W1 Booking Details call on open: bookingIds* (max 40) → status*, checkIn*/checkOut*, cancellationPolicy*, hotelComNumber.

W2 Preview is fetched only on entry to review: bookingId* → reference*, policyText*, refundRate{currency*, inclusive*}, SLA 120 s.

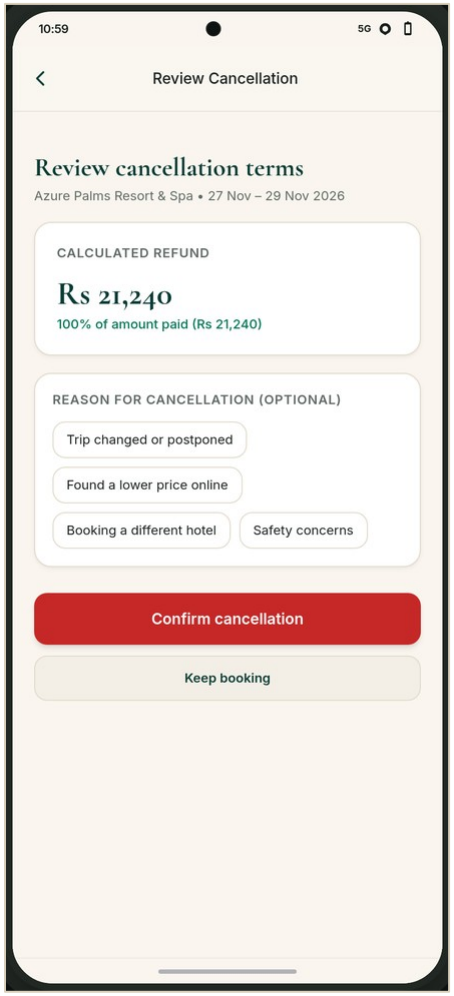
BUSINESS

Both Hypothesis to measure: self-serve state visibility should cut 'what is my status' support volume.

Scenario data (synthetic): Azure Palms Resort & Spa · booking 80000610 · Rs 21,240

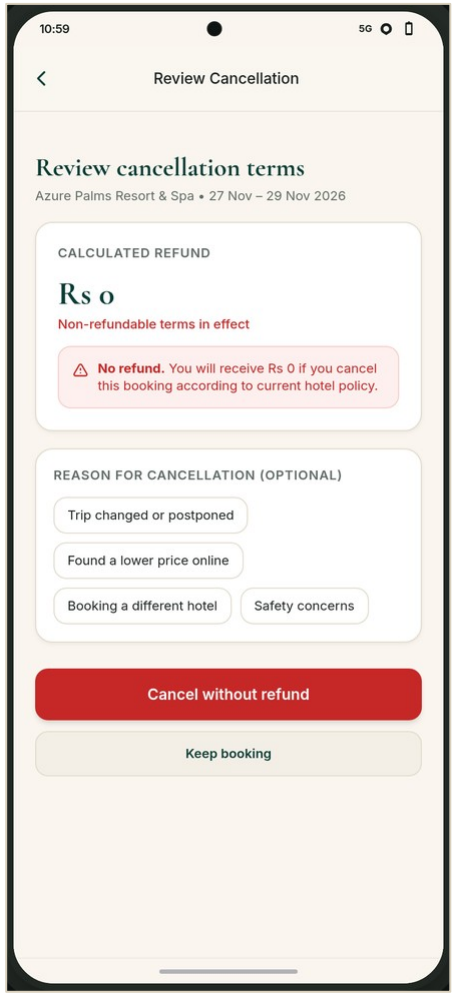
Wireframe 3: Review cancellation (standard / zero)

The customer confirms the number, not just the action.



W3

Review - standard refund



W3·0

Review - zero refund

DESIGNER

Std Visual hierarchy - refund amount and policy outrank decoration: the customer confirms the number.

Zero Consent in plain words - the button itself says 'Cancel without refund': zero is clear.

Both Error prevention - 'Keep booking' stays easy to find.

DEVELOPER

Std Preview within the 120 s SLA; refundRate is a money object {currency*, inclusive*}, inclusive = amount incl. taxes, not a percentage.

Both Confirm sends bookingId*, the latest reference* and the matching refundRate{currency*, inclusive*}; cancelReason* is required - the UI reason is optional, documented 0 = None.

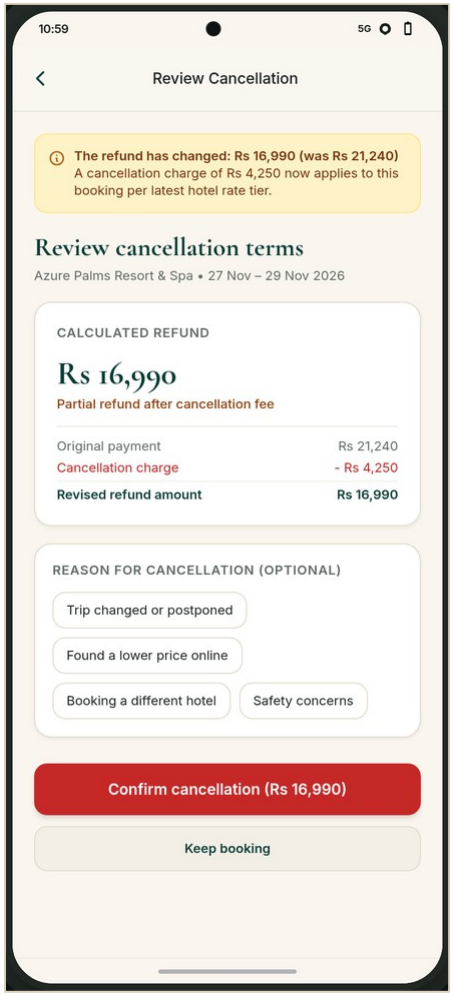
BUSINESS

Both Hypothesis to measure: informed consent should reduce refund disputes.

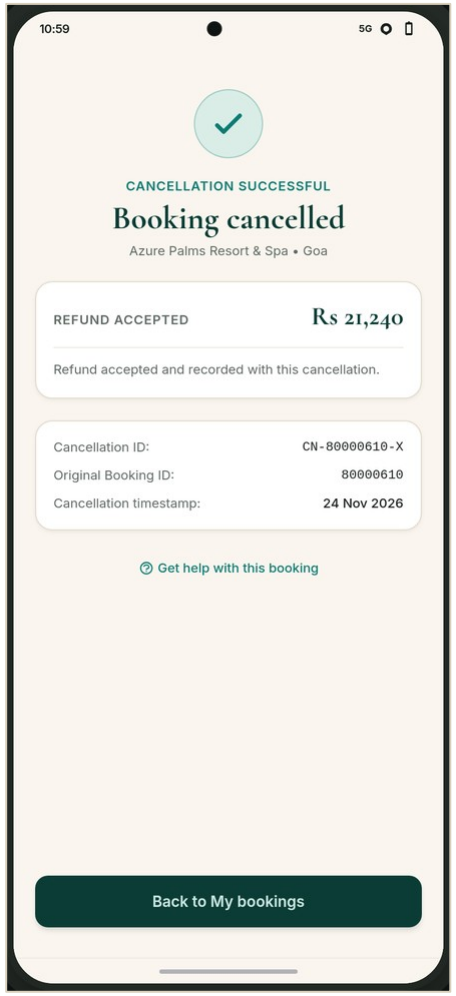
Scenario data (synthetic): Rs 21,240 standard · Rs 0 zero-refund

Wireframes 3b-5: Changed refund → fresh consent

If the amount moves, we ask again - and the receipt says exactly what happened.



W3b
Changed refund - re-consent



W5
Cancellation receipt

DESIGNER

W3b Error prevention via fresh consent - the banner names old and new amounts, confirmed in the button.

W5 Honest feedback - the receipt confirms cancellation without promising payout timing or method.

DEVELOPER

W3b error 914 (refund mismatch) → re-preview → re-confirm with the new reference*.

W5 Success is HTTP 200 with an empty body; the receipt records reference* and timestamp.

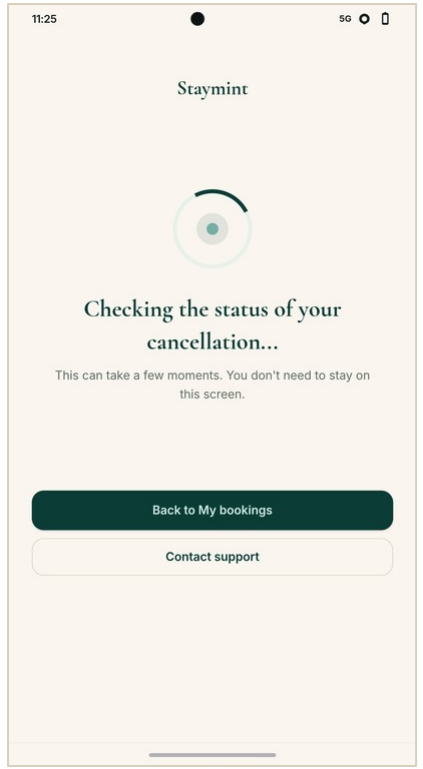
BUSINESS

Both Hypothesis to measure: re-consent keeps consent valid when terms move, protecting dispute rates.

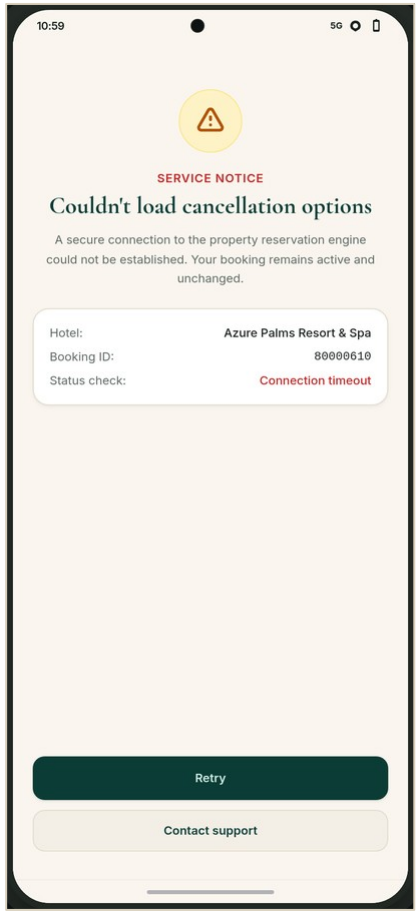
Scenario data (synthetic): Rs 21,240 → Rs 16,990 · fee Rs 4,250

Wireframes 4-6: Waiting, failure, and 'still active'

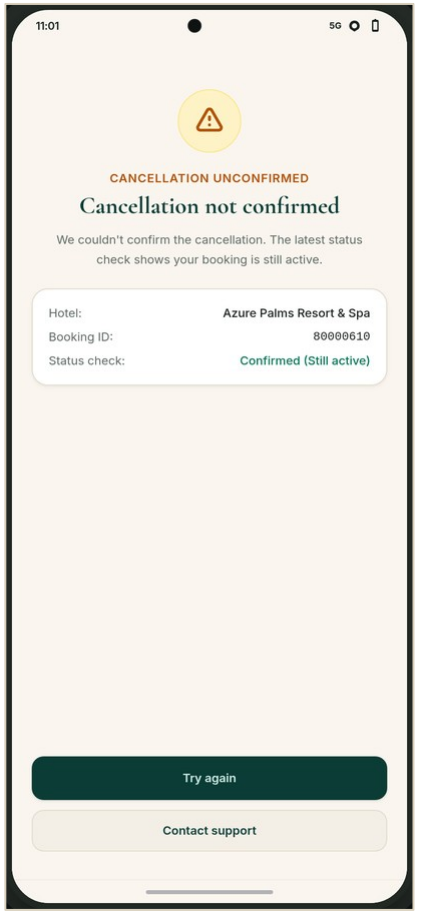
An uncertain cancellation is never presented as success.



W4
Checking latest status



W6a
Preview failed - unchanged



W6b
Still active - safe to retry

DESIGNER

All Visibility of system status - each state is visually distinct.

W4 Preserved context - waiting keeps the thread: leave and return.

DEVELOPER

W4 Confirm timeout → recheck via Booking Details (>=3 tries / 15 min) before any claim.

W6a A failed preview is separated from an ineligible booking.

W6b The latest status check shows the booking is still active.

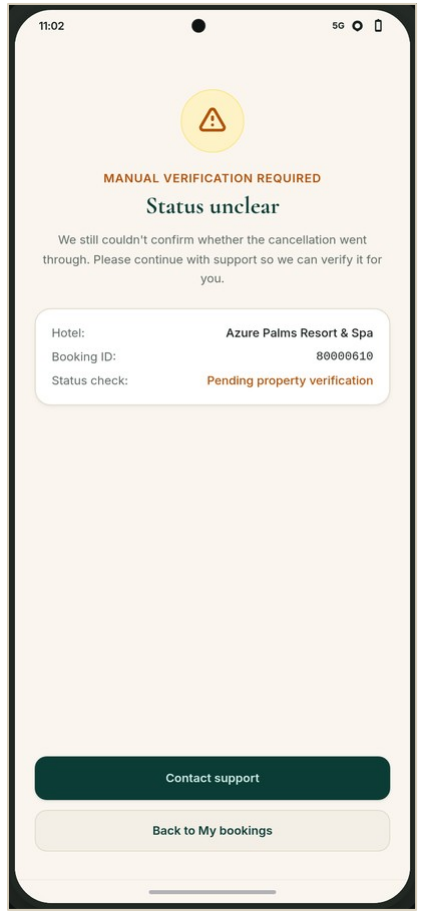
BUSINESS

All Hypothesis to measure: honest states preserve rebooking confidence.

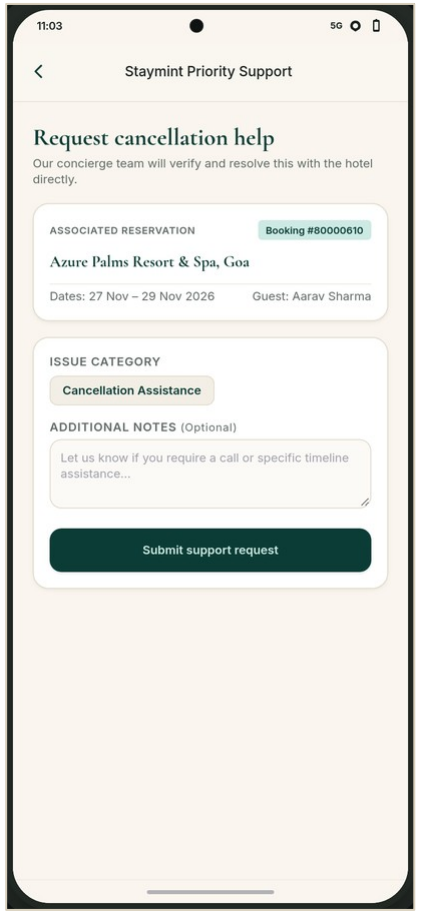
Scenario data (synthetic): status checks are simulated

Wireframes 6c-7: Support with context, and clear no's

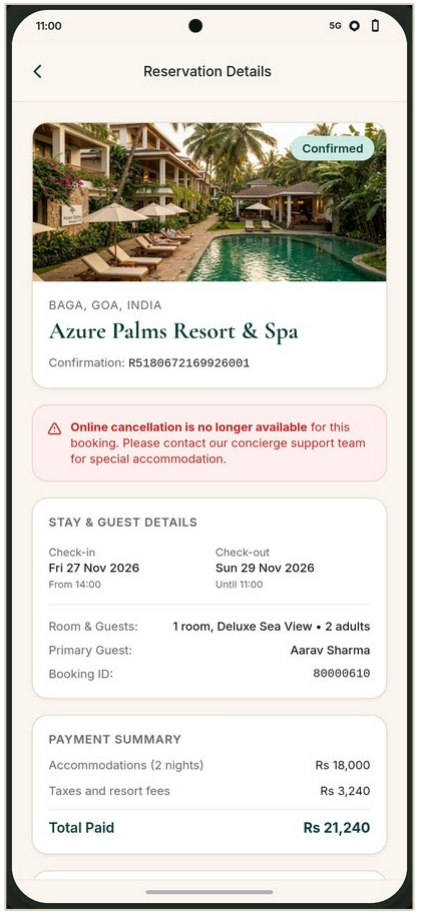
When the app cannot resolve it, support starts where the customer left off.



W6c
Status unclear - verify



W7
Support request - prefilled



W2b
Cancellation unavailable

DESIGNER

W7 Reduced repetition - booking context is prefilled for support.

W2b Clear terminal state - a clear no, not a dead end or retry loop.

DEVELOPER

W7 Support request carries the booking reference and a status snapshot; is internal, not an Agoda call.

W2b Unavailable is separated from failure: no retry loop offered.

BUSINESS

Both Hypothesis to measure: context-carrying support should resolve faster with less repeat explanation.

Scenario data (synthetic): support flow is simulated

Run it live

Every screen in this deck is running code you can click through.

INTERACTIVE PROTOTYPE - SYNTHETIC DATA

staymint-prototype.vercel.app

Open in any modern browser - no sign-in, no backend.

HOW TO REVIEW

1 · Open the link above.
2 · Pick a reviewer scenario.
3 · Click through: every state in Slides 8-12 is reachable.

THE 9 REVIEWER SCENARIOS

1 Standard (Rs 21,240)
2 Zero refund (Rs 0)
3 Updated refund (Rs 16,990)
4 Timeout resolved (W4C → W5)
5 Timeout active (W4C → W6b)
6 Timeout unresolved (W4C → W6c)
7 Preview failure (W4A → W6a)
8 Already cancelled
9 Cancellation unavailable

The flow is not just drawn - it runs.

Deterministic synthetic data: Azure Palms Resort & Spa · booking 80000610 · not connected to live Agoda systems.